

AT32 PMSM FOC Magnetic Encoder Demo

Introduction

This document introduces how to use AT-MOTOR-EVB evaluation board equipped with FOC vector control algorithm to demonstrate how to drive a three-phase permanent magnet synchronous motor (BLDC) with a magnetic encoder, including the environment setup, program document architecture, macro definitions, and state machine procedures. In this demo, AT32F413RCT7 is selected as an embedded transfer board.

Applicable products:

Part number	AT32F402/405 series
	AT32F403A/407 series
	AT32F413 series
	AT32F415 series
	AT32F421 series
	AT32F422/426 series
	AT32F425 series
	AT32F423 series
	AT32F435/437 series
	AT32F45x series
	AT32M412/M416 series

Contents

1	Motor control library algorithms	5
2	Environment requirements	6
	2.1 Hardware requirements	6
	2.2 Software requirements	8
3	PMSM motor control library documents	12
4	State machine	24
	4.1 Description of state	24
5	How to use demo project	27
6	Revision history	28

List of tables

Table 1. Motor specifications	6
Table 2. Magnetic encoder specifications	7
Table 3. Correspondence between Flash size and ROM	8
Table 4. Description of motor control library documents	13
Table 5. Drive mode definitions	15
Table 6. Control parameter definitions	16
Table 7. Driver parameter definitions	18
Table 8. Motor parameter definitions.....	20
Table 9. Configuration of peripherals (AT32F413RCT7)	21
Table 10. Peripheral configuration related functions.....	22
Table 11. Motor control related interrupt functions (AT32F413RCT7).....	22
Table 12. Initial state and state switching conditions	25
Table 13. Document revision history	28

List of figures

Figure 1. 2808-KV630 motor	6
Figure 2. AT-MOTOR-EVB V2.x evaluation board.....	7
Figure 3. AT32F413RCT7 ROM settings (Keil).....	9
Figure 4. AT32F413RCT7 ROM settings (AT32IDE)	10
Figure 5. AT32F413RCT7 ROM settings (IAR).....	11
Figure 6. Structure of motor control library documents	12
Figure 7. Structure of motor control project.....	13
Figure 8. Encoder ABZ signal relationship	20
Figure 9. State machine process flow	24

1 Motor control library algorithms

This document contains the following motor control algorithms:

Target motor type: Three-phase permanent magnet synchronous motor (BLDC)

Control modes:

- FOC vector control

Three-phase PWM method:

- SVPWM

Phase current sensing methods:

- 3-shunt current sensing
- 2-shunt current sensing
- 1-shunt current sensing and reconstruction method

Rotor position detection mode:

- Magnetic (absolute) encoder

Sensor-based FOC sinusoidal control mode:

- Voltage vector control
- Torque control (current vector control)
- Speed control
- Field weakening control
- Positioning control

Auto tuning functions:

- Motor winding parameters identification
- Current PI control parameters self-tuning
- Magnetic encoder deviation angle self-calibration

2 Environment requirements

2.1 Hardware requirements

Hardware resources required include a PMSM (BLDC) motor with a magnetic encoder, AT-Link or third-party debugger and a motor control evaluation board (AT-MOTOR-EVB). For details about hardware configurations, refer to *UM0014AT32_LV_Motor_Control_EVB_User_Manual_V2.x*.

- PMSM (BLDC) motor: 2808-KV630

Figure 1. 2808-KV630 motor

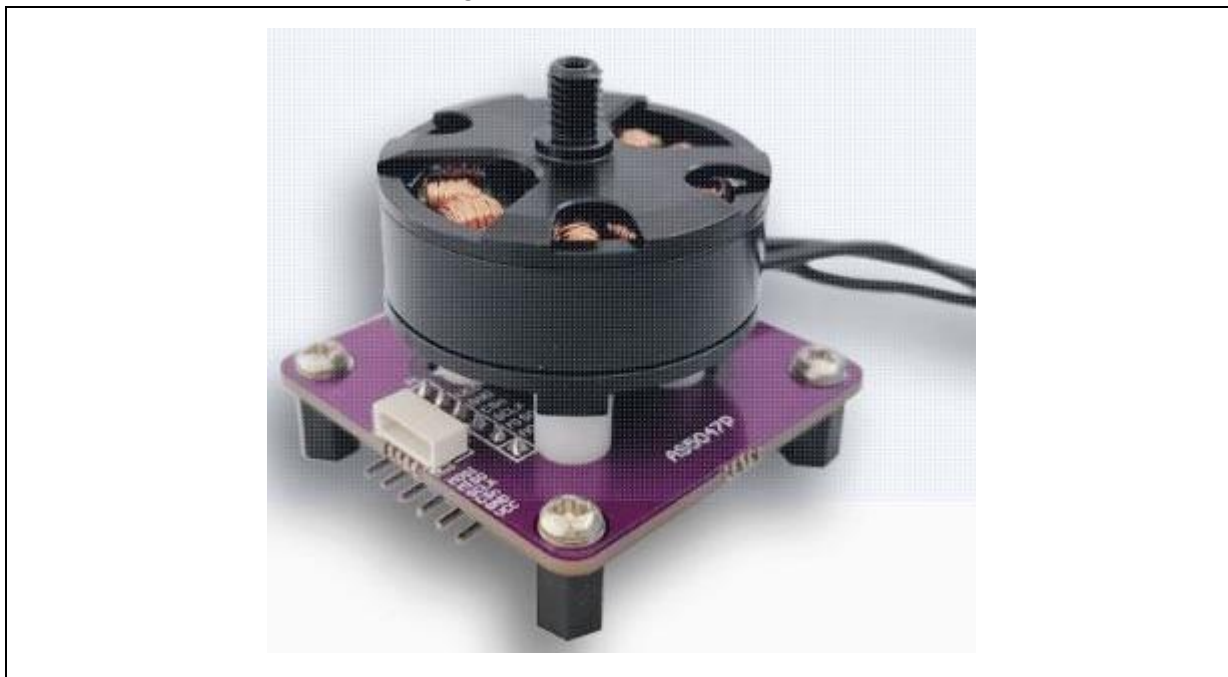


Table 1. Motor specifications

Model	2808-KV630
Number of poles	7
Rated voltage	DC 12V
Maximum speed	7500 RPM
Rated torque	0.1 N·m
Maximum current	10 A
Armature resistance	1.4Ω
Armature inductance	25uH
Speed constant (Kv)	630Kv
Flux linkage	0.00125
Weight	59g

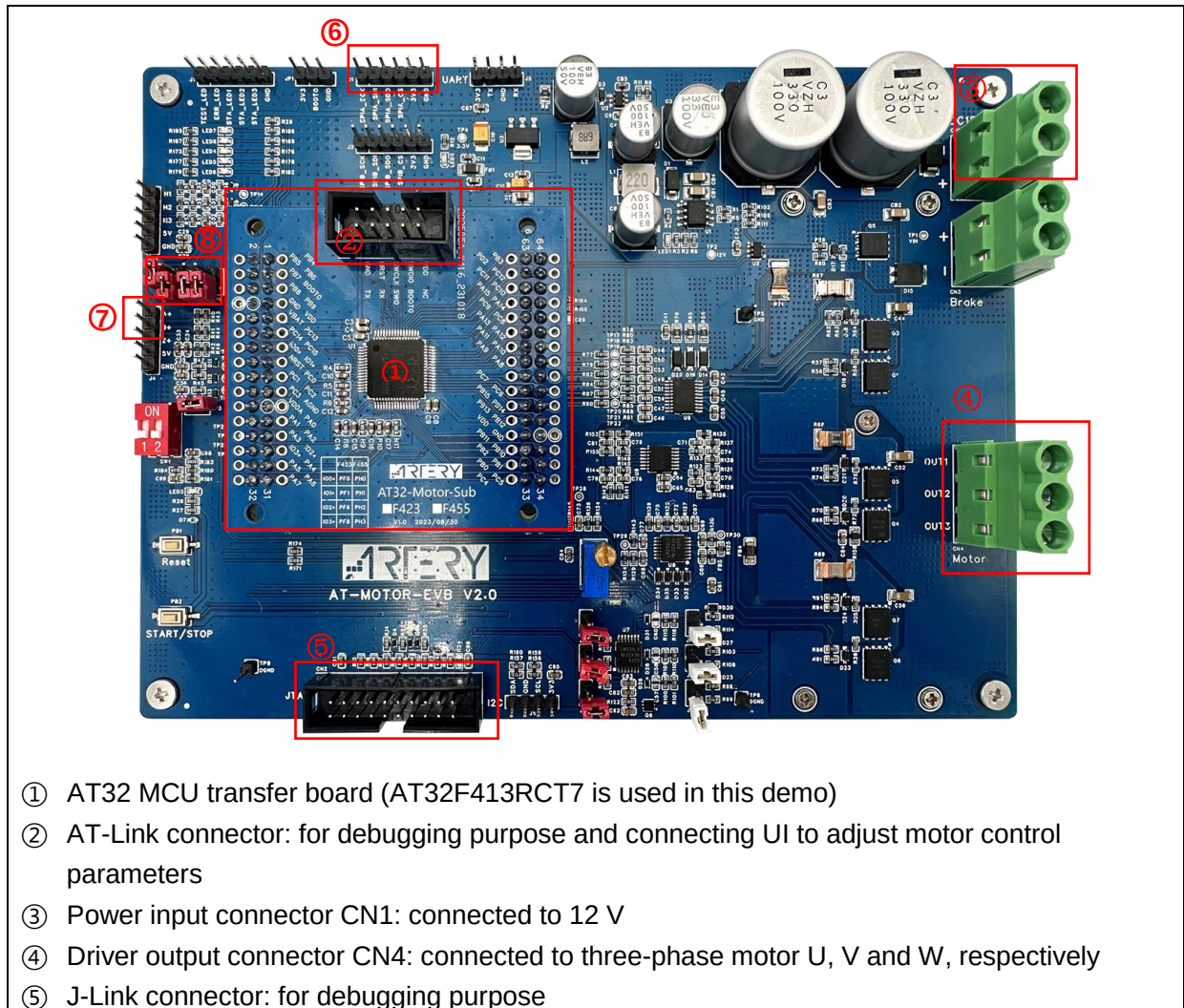
- Magnetic encoder: ams OSRAM AS5047P

Table 2. Magnetic encoder specifications

Model	AS5047P
Interface 1	SPI
Resolution (unit: bit)	14
Interface 2	ABZ
ABZ CPR	4096 (default: 4000)
Operating voltage	3.3-5 V
Operating current	15 mA
Package	TSSOP-14
High speed application	Up to 28krpm
Digital interface SCLK maximum frequency	10 MHz

- Debugger
- Motor control board: AT-MOTOR-EVB V2.x

Figure 2. AT-MOTOR-EVB V2.x evaluation board



- ⑥ Magnetic encoder SPI interface J1: connected with encoder according to instruction (note [1])
- ⑦ Magnetic encoder ABZ interface J4: connected with A, B, Z according to instruction
- ⑧ Jumper for F422、F421 and L021 AT32-Motor-Sub board
M_METHOD: JP2 CLOSED; JP4, JP5 and JP6 CLOSED
MT_METHOD: NA (Note[4])

Jumper for other AT32-Motor-Sub board

M_METHOD: JP2 CLOSED; JP4, JP5 and JP6 OPEN

MT_METHOD: JP2, JP4 and JP5 CLOSED; JP6 OPEN

Note [1]: When SPIA connector is used, remove C16, C17 and C18 capacitors from AT32-MOTOR-Sub and remove R119, R120, R121 and R25 resistors from AT-MOTOR-EVB V2.0 board or remove JP13, JP14 and JP15 jumpers from AT-MOTOR-EVB V2.1 board.

2.2 Software requirements

- 1) Take the AT32F413 as an example. Open the demo project under AT32F413_MC_Library_Project\motor_evb_2v0\at32f413\pmsm_foc_magnetic_encoder (note[2]).
- 2) Run ArteryMotorMonitor.exe directly (installation not required).
- 3) In Keil IDE, go to **Options - Read/Only Memory Areas** to set ROM according to the Flash memory size of each AT32 MCU, as shown in Table 3 (note [3]). For example, for AT32F413RCT7 with a 256 KB Flash memory, its IROM1 start address is 0x8000000 with a size of 0x3F000, whereas its IROM2 start address is 0x803F000 with a size of 0x1000 (see Figure 3). In AT32IDE, modify *ld file according to the Flash memory size of each AT32 MCU (see Figure 4). In IAR Systems, modify *icf file according to the Flash memory size of each AT32 MCU (see Figure 5).

Table 3. Correspondence between Flash size and ROM

Flash size	4032K	1024K	512K	448K	256K
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x3EE000	0xFF000	0x7F000	0x6E000	0x3F000
IROM2(address)	0x83EE000	0x80FF000	0x807F000	0x0806E000	0x803F000
IROM2(size)	0x2000	0x1000	0x1000	0x2000	0x1000

Flash size	128K	64K	32K	16K
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x1F400	0x0F400	0x07400	0x03400
IROM2(address)	0x801F400	0x800F400	0x8007400	0x8003400
IROM2(size)	0xC00	0xC00	0xC00	0xC00

Note [2]: This demo project does not support Keil compiler version 5 and O0 optimization level for compiling simultaneously. Please use Keil compiler version 6 or O1 optimization level for compiling. In case of Keil_v5.33, since AT32 BSP source code does not support V6.15 compiler, the Keil compiler version 5 and O1 optimization

level should be used.

Note [3]: The magnetic encoder demo project reserves a Flash memory for magnetic encoder deviation angle calibration lookup table; therefore, the correspondence between Flash size and ROM is different from that of other demo projects.

Figure 3. AT32F413RCT7 ROM settings (Keil)

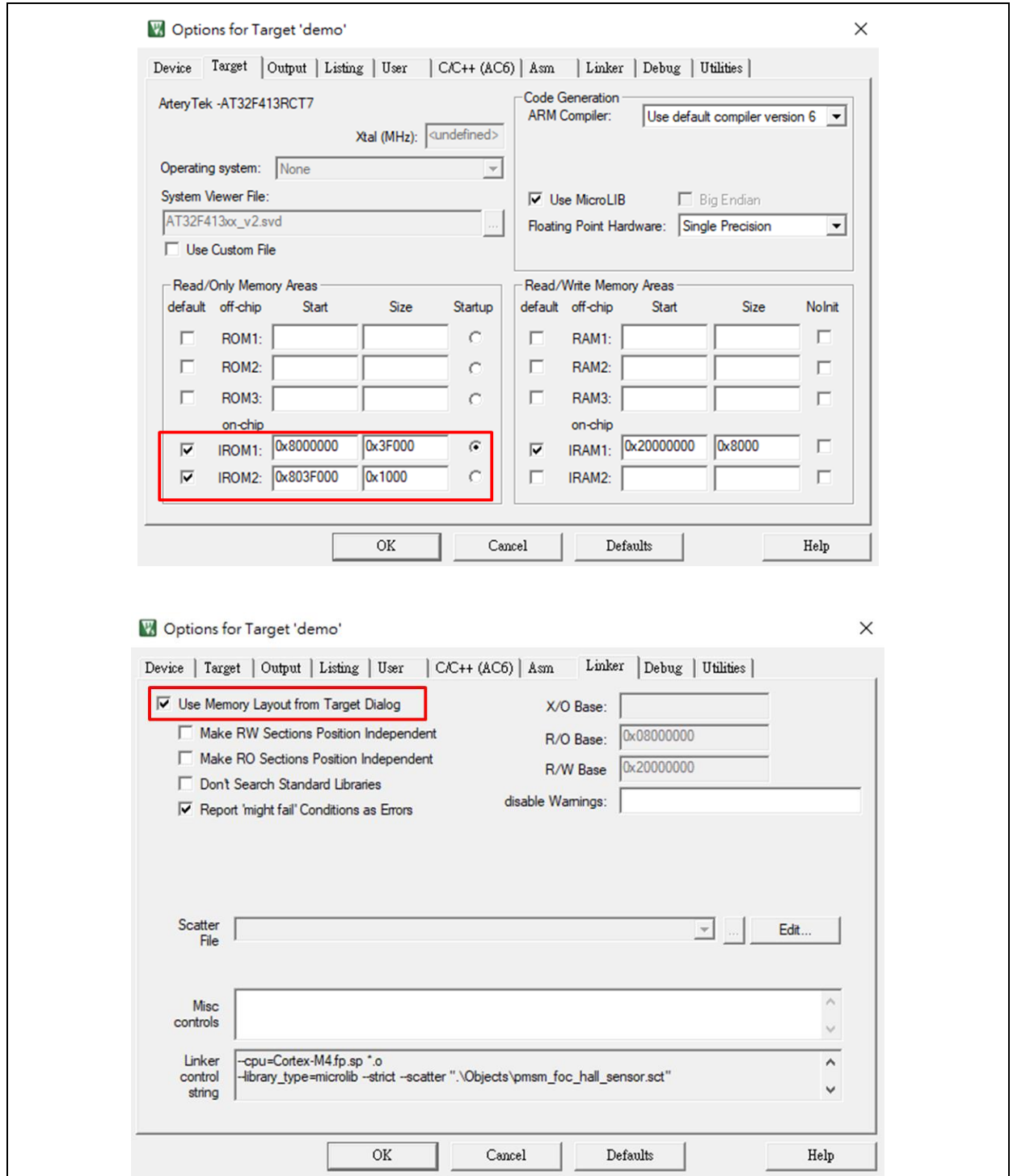


Figure 4. AT32F413RCT7 ROM settings (AT32IDE)

```

25 _estack = 0x20008000; /* end of RAM */
26
27 /* Generate a link error if heap and stack don't fit into RAM */
28 _Min_Heap_Size = 0x200; /* required amount of heap */
29 _Min_Stack_Size = 0x400; /* required amount of stack */
30
31 /* Specify the memory areas */
32 MEMORY
33 {
34 FLASH (rx)      : ORIGIN = 0x08000000, LENGTH = 0x3F000
35 MC_DATA (r)      : ORIGIN = 0x0803F000, LENGTH = 0x1000
36 RAM (xrw)       : ORIGIN = 0x20000000, LENGTH = 32K
37 }
38
39 /* Define output sections */
40 SECTIONS
41 {
42 /* The startup code goes first into FLASH */
43 .isr_vector :
44 {
45     . = ALIGN(4);
46     KEEP(*(.isr_vector)) /* Startup code */
47     . = ALIGN(4);
48 } >FLASH
49
50 .mc_data :
51 {
52     . = ALIGN(4);
53     . = ORIGIN(MC_DATA);
54     MC_EncoderStoreAddr = .;
55     *(.MC_EncoderStoreAddr);
56     . = ALIGN(4);
57     . = ORIGIN(MC_DATA) + 0x800;
58     _MC_VectStoreAddr1 = .;
59     *(.MC_VectStoreAddr1);
60     . = ALIGN(4);
61     . = ORIGIN(MC_DATA) + 0x800 + 800;
62     _MC_VectStoreAddr2 = .;
63     *(.MC_VectStoreAddr2);
64     . = ALIGN(4);
65 } >MC_DATA
66
67 /* The program code and other data goes into FLASH */
68 .text :
69 {
70     . = ALIGN(4);
71     *(.text)           /* .text sections (code) */
72     *(.text*)          /* .text* sections (code) */
73     *(.glue_7)         /* glue arm to thumb code */
74     *(.glue_7t)        /* glue thumb to arm code */
75     *(.eh_frame)

```

Figure 5. AT32F413RCT7 ROM settings (IAR)

```

1  /****ICF**** Section handled by ICF editor, don't touch! ****/
2  /*-Editor annotation file-*/
3  /* IcfEditorFile="$TOOLKIT_DIR$\config\ide\IcfEditor\cortex_v1_0.xml" */
4  /*-Specials-*/
5  define symbol __ICFEDIT_intvec_start__ = 0x08000000;
6  /*-Memory Regions-*/
7  define symbol __ICFEDIT_region_ROM_start__ = 0x08000000;
8  define symbol __ICFEDIT_region_ROM_end__ = 0x0803F000 - 1;
9  define symbol __ICFEDIT_region_ROM1_start__ = 0x0803F000;
10 define symbol __ICFEDIT_region_ROM1_end__ = (__ICFEDIT_region_ROM1_start__ + 0x800 - 1);
11 define symbol __ICFEDIT_region_ROM2_start__ = (__ICFEDIT_region_ROM1_end__ + 1);
12 define symbol __ICFEDIT_region_ROM2_end__ = (__ICFEDIT_region_ROM2_start__ + 800 - 1);
13 define symbol __ICFEDIT_region_ROM3_start__ = (__ICFEDIT_region_ROM2_end__ + 1);
14 define symbol __ICFEDIT_region_ROM3_end__ = 0x0803F000 + 0x1000 - 1;
15 define symbol __ICFEDIT_region_RAM_start__ = 0x20000000;
16 define symbol __ICFEDIT_region_RAM_end__ = 0x20007FFF;
17 /*-Sizes-*/
18 define symbol __ICFEDIT_size_cstack__ = 0x400;
19 define symbol __ICFEDIT_size_heap__ = 0x200;
20 /**** End of ICF editor section. ****ICF****/
21
22 define memory mem with size = 4G;
23 define region ROM region = mem: [from __ICFEDIT_region_ROM_start__ to __ICFEDIT_region_ROM_end__];
24 define region ROM1 region = mem: [from __ICFEDIT_region_ROM1_start__ to __ICFEDIT_region_ROM1_end__];
25 define region ROM2 region = mem: [from __ICFEDIT_region_ROM2_start__ to __ICFEDIT_region_ROM2_end__];
26 define region ROM3 region = mem: [from __ICFEDIT_region_ROM3_start__ to __ICFEDIT_region_ROM3_end__];
27 define region RAM region = mem: [from __ICFEDIT_region_RAM_start__ to __ICFEDIT_region_RAM_end__];
28
29 define block CSTACK with alignment = 8, size = __ICFEDIT_size_cstack__ { };
30 define block HEAP with alignment = 8, size = __ICFEDIT_size_heap__ { };
31
32 initialize by copy { .readwrite };
33 do not initialize { .section .noinit };
34
35 place at address mem: __ICFEDIT_intvec_start__ { .readonly section .intvec };
36
37 place in ROM region { .readonly };
38 place in ROM1 region { .readonly section .MC_EncoderStoreAddr };
39 place in ROM2 region { .readonly section .MC_VectStoreAddr1 };
40 place in ROM3 region { .readonly section .MC_VectStoreAddr2 };
41 place in RAM region { .readwrite,
42 ..... block CSTACK, block HEAP };

```

3 PMSM motor control library documents

Figure 6 shows the structure of motor control library documents and their relationships. The *motor_control_drive_param.h* file contains the motor control drive modes, motor parameters, control board parameters and controller parameters to be set by users. Users can set MCU peripheral parameters in *mc_hwio.h* file according to the connection relations between MCU peripherals and control board circuits. Relevant setting parameters are integrated in *mc_foc_globals.h* (*mc_bldc_globals.h*), and then their initial variables are set by functions in *mc_foc_globals.c* (*mc_bldc_globals.c*) and used by motor control library functions. For the MCU peripherals initialization, it is executed in *mc_hwio.c*.

Figure 6. Structure of motor control library documents

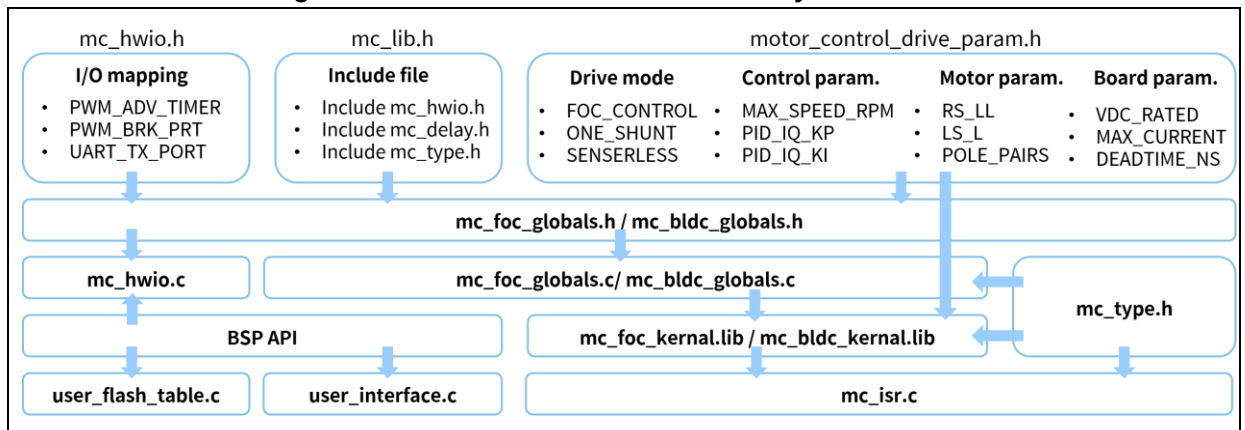


Figure 7 shows the document structure of the demo project. The “**user**” folder is user-defined and it contains the main program, peripheral program, writing parameters table into Flash, and parameter definition header files. The “**firmware**” folder contains BSP files, and the “**mcilib**” folder contains motor control library program, including delay functions, communication functions, global variable setting functions and motor control library functions. For details about motor control library functions, refer to AN0064 *AT32_Motor_Control_Library_User_Guide*.

Figure 7. Structure of motor control project

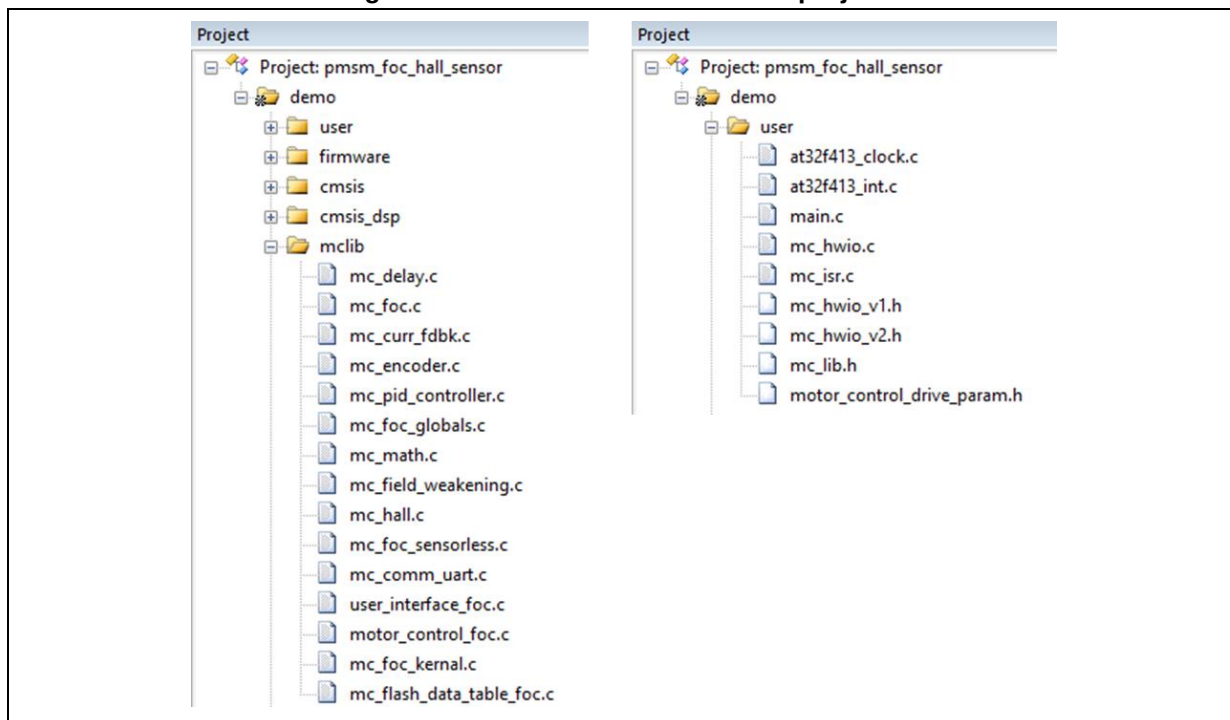


Table 4 lists motor control library documents.

Table 4. Description of motor control library documents

File name	Description
mc_lib.h	Unified header file management
motor_control_drive_param.h	Set motor drive modes (current sampling mode, sensor mode etc.), control parameters, driver parameters, motor parameters
mc_hwio.c	Hardware peripheral configuration
mc_hwio_v1.h	Hardware IO macro definitions (AT-MOTOR-EVB V1.x)
mc_hwio_v2.h	Hardware IO macro definitions (AT-MOTOR-EVB V2.x)
mc_isr.c	Relevant motor control interrupt functions
mc_flash_data_table_foc.c	Writing parameters table into Flash
mc_flash_data_table_foc.h	Relevant configuration for writing parameters table into Flash
mc_type.h	Global variables types and definitions, enumeration definition
mc_delay.c	Delay-related functions
mc_delay.h	Declaration of delay-related functions
mc_comm_uart.c	Configuration of peripherals related to communication interface
mc_comm_uart.h	Declaration and configuration of communication UART related functions
mc_pid_control.c	PID controller related functions
mc_pid_control.h	Declaration of PID controller related functions
mc_curr_fdbk.c	Current sensing related functions
mc_curr_fdbk.h	Declaration of current sensing related functions
mc_math.c	Filter related functions

mc_math.h	Declaration of filter related functions
mc_hall.c	Hall sensor related functions
mc_hall.h	Declaration of Hall sensor related functions
mc_foc_kernal.lib	Motor control library core functions (for Keil environment only) (applicable to M4F MCU)
mc_foc_kernal_noFPU.lib	Motor control library core functions (for Keil environment only) (applicable to M4 MCU)
mc_foc_kernal_m0plus.lib	Motor control library core functions (for Keil environment only) (applicable to M0+ MCU)
libmc_foc_kernal.a	Motor control library core functions (for AT32IDE environment only) (applicable to M4F MCU)
libmc_foc_kernal_noFPU.a	Motor control library core functions (for AT32IDE environment only) (applicable to M4 MCU)
libmc_foc_kernal_m0plus.a	Motor control library core functions (for AT32IDE environment only) (applicable to M0+ MCU)
mc_foc_kernal.a	Motor control library core functions (for IAR system only) (applicable to M4F MCU)
mc_foc_kernal_noFPU.a	Motor control library core functions (for IAR system only) (applicable to M4 MCU)
mc_foc_kernal_m0plus.a	Motor control library core functions (for IAR system only) (applicable to M0+ MCU)
mc_foc_kernal.h	Declaration of motor control library kernel functions
motor_control_foc.c	Motor control related functions
motor_control_foc.h	Declaration of motor control related functions
mc_foc.c	Vector control related functions
mc_foc.h	Declaration of vector control related functions
mc_encoder.c	Encoder related functions
mc_encoder.h	Declaration of encoder related functions
mc_field_weakening.c	Filed weakening related functions
mc_field_weakening.h	Declaration of filed weakening related functions
mc_foc_sensorless.c	Sensorless mode related functions
mc_foc_sensorless.h	Declaration of sensorless mode related functions
mc_curr_decoupling.c	Current decouple control related functions
mc_curr_decoupling.h	Declaration of current decouple control related functions
MTPA_MTPV_table.c	Maximum Torque Per Ampere (MTPA)/ Maximum Torque Per Volt (MTPV) field-weakening lookup table function
MTPA_MTPV_table.h	Declaration of Maximum Torque Per Ampere (MTPA)/ Maximum Torque Per Volt (MTPV) field-weakening lookup table function
mc_foc_globals.c	Global variables definition and default values, global functions declaration

mc_foc_globals.h	Global variables, global function declaration, macro definitions
user_interface_foc.c	Functions related to communication interface
user_interface_foc.h	Declaration of functions related to communication interface
AS5047P.h	Declaration of magnetic encoder (AS5047P) SPI interface macro definition and related functions
AS5047P.c	Magnetic encoder (AS5047P) SPI interface configuration and related functions

1) **motor_control_drive_param.h** file

- This file is divided into four parts, which are used to configure drive mode, control parameters, motor parameters and board parameters.
- Table 5 lists definitions of drive modes that can be set according to the hardware and motor configuration. In this demo, the magnetic encoder with SPI interface is selected, and the ABZ interface is configurable according to the magnetic encoder. If ABZ interface is configured, MT_METHOD or M_METHOD method can be selected for speed measurement (note [4]). For current sampling method, it is possible to select 3-shunt, 2-shunt or 1-shunt current sensing mode according to hardware configurations. In addition, the field weakening control is optional according to users' needs.

Table 5. Drive mode definitions

Definition	Description
FOC_CONTROL	FOC vector control
THREE_SHUNT	Three-shunt current sampling
TWO_SHUNT	Two-shunt current sampling
ONE_SHUNT	Single-shunt current sampling
U_V_SHUNT	U & V shunt (for TWO_SHUNT only)
V_W_SHUNT	V & W shunt (for TWO_SHUNT only)
U_W_SHUNT	U & W shunt (for TWO_SHUNT only)
AT_MOTOR_EVB_V2	Applicable to AT-MOTOR_EVB V2.x
GATE_DRIVER_LOW_SIDE_INVERT	Enable low-side MOS inverted output
MAGNET_ENCODER_W_ABZ	Magnetic encoder SPI communication (with ABZ interface)
MAGNET_ENCODER_WO_ABZ	Magnetic encoder SPI communication (without ABZ interface)
REVERSE_ENCODER_COUNT	Reverse encoder counting (magnetic encoder ABZ interface)
M_METHOD	M method for speed measurement (magnetic encoder ABZ interface) (note [4])
MT_METHOD	M/T method for speed measurement (magnetic encoder ABZ interface) (note [4])
ANGLE_CALIBRATION	Magnetic encoder angle self-calibration
FIELD_WEAKENING	Field weakening control
CURRENT_LP_FILTER	Get d/q axis current low-pass filter signals (vector control)

AC_CURRENT_LP_FILTER	Get A/B/C Phase Low-Pass Filtered Current Signals (Vector Control/FOC)
INTERNAL_CLOCK_SOURCE	Use MCU internal oscillator as a clock source
MOTOR_PARAM_IDENTIFY	Motor winding parameters identification
CURRENT_DECOUPLE_CTRL	Current decouple control
TORQUE_CTRL_WITH_SPEED_LIMIT	Torque (current) control, including the maximum speed limitation
BRAKING_RESISTOR	Use an external braking resistor
SPMSM	Surface-mounted permanent magnet synchronous motor
IPMSM	Interior permanent magnet synchronous motor
IPM_MTPA_MTPV_TABLE	MTPA/MTPV lookup table (for IPMSM only)
IPM_MTPA_MTPV_CTRL	MTPA/MTPV lookup table control (for IPMSM only)
USE_MOTOR_MONITOR	Artery motor UI tool
MOS_RDS_SHUNT	Current Sampling Utilizing Low-Side MOSFET On-Resistance
TWO_ADC_CONVERTERS	Current Sampling with Dual-ADC Synchronization (Applicable for MCUs with dual or more ADCs; Not compatible with ONE_SHUNT topology)

Note [4]: *M_METHOD/MT_METHOD* are speed measurement methods. In most cases, the *M_METHOD* method is selected as this method only needs to capture the number of pulse of the encoder. For higher accuracy, the *MT_METHOD* method can be used, which requires an extra timer to obtain the encoder pulse interval time in addition to the pulse number. In AT-MOTOR-EVB, it uses JP4 and JP5 CLOSED to connect with a timer to implement *MT_METHOD* settings.

- Table 6 lists the definitions of control parameters that can be defined according to the hardware, motor specifications and control characteristics. Some parameters are configurable, such as PI controllers of d-axis and q-axis currents, speed PI controller, etc.

Table 6. Control parameter definitions

Definition	Description
PWM_FREQ	PWM output frequency (unit: Hz)
MOTOR_CONTROL_MODE	Motor control mode (speed control, torque control, etc.; refer to motor_control_mode in <i>type.h</i> for details)
CTRL_SOURCE	Control source settings (external source control/software control)
UI_UART_BAUDRATE	Baud rate of UI communication serial port
TUNE_TARGET_CURRENT	Tune target current value of PI parameters (unit: ampere)
TUNE_CURRENT_TOTAL_PERIOD	Tune total current pulse period of PI parameters (unit: ms)
TUNE_CURRENT_STEP_PERIOD	Tune current pulse step period of PI parameters (unit: ms)
SPEED_LOOP_FREQ	Speed loop frequency (unit: Hz)

Definition	Description
MIN_SPEED_RPM	Minimum motor speed (unit: rpm)
MAX_SPEED_RPM	Maximum motor speed (unit: rpm)
MIN_POSCTL_SPD	Minimum speed for position loop control (unit: rpm)
ACC_SPD_SLOPE	Slope of acceleration (unit: rpm/ms, when systick =1kHz)
DEC_SPD_SLOPE	Slope of deceleration (unit: rpm/ms, when systick =1kHz)
SP_MAX_VOLT	Maximum voltage of external command source (unit: voltage)
SP_THRESHOLD	Threshold voltage of external command source (unit: voltage)
SP_RUN_VALUE	Minimum voltage to start running in external source mode (unit: voltage)
SP_STOP_VALUE	Maximum voltage to stop running in external source mode (unit: voltage)
PID_SPD_KP_DIV	Speed control proportional gain divisor (Q16 mode)
PID_SPD_KI_DIV	Speed control integral gain divisor (Q16 mode)
PID_SPD_KP_DEFAULT	Speed control proportional gain (Q15 mode)
PID_SPD_KI_DEFAULT	Speed control integral gain (Q15 mode)
POSITION_LOOP_FREQ	Position loop control frequency (unit: Hz)
MAX_POSITION_ANGLE	Maximum position angle (unit: Degree)
MIN_POSITION_ANGLE	Minimum position angle (unit: Degree)
CMD_TO_VAL_GAP	Define the counter gap value between rotor position and its target to use PID_POS_KI_DEFAULT_STABLE
SMALL_POS_CMD_GAP	When the rotor position difference is within a certain range of the target position, the minimum speed command (MIN_POSCTL_SPD) is used for position loop control
PID_POS_KP_DEFAULT	Rotor position control proportional gain (Q15 mode)
PID_POS_KI_DEFAULT	Rotor position control integral gain (Q15 mode)
PID_POS_KI_DEFAULT_STABLE	Rotor position control integral gain (Q15 mode) (rotor position close to target position)
PID_POS_KD_DEFAULT	Rotor position control derivative gain (Q15 mode)
PID_POS_KP_GAIN_DIV	Rotor position control proportional gain divisor (Q16 mode)
PID_POS_KI_GAIN_DIV	Rotor position control integral gain divisor (Q16 mode)
PID_POS_KD_GAIN_DIV	Rotor position control derivative gain divisor (Q16 mode)
PID_ID_KP_DEFAULT	d-axis current control proportional gain (Q15 mode)
PID_ID_KI_DEFAULT	d-axis current control integral gain (Q15 mode)
PID_ID_KP_GAIN_DIV	d-axis current control proportional gain divisor (Q16 mode)
PID_ID_KI_GAIN_DIV	d-axis current control integral gain divisor (Q16 mode)
PID_IQ_KP_DEFAULT	q-axis current control proportional gain (Q15 mode)
PID_IQ_KI_DEFAULT	q-axis current control integral gain (Q15 mode)

Definition	Description
PID_IQ_KP_GAIN_DIV	q-axis current control proportional gain divisor (Q16 mode)
PID_IQ_KI_GAIN_DIV	q-axis current control integral gain divisor (Q16 mode)
AUTO_TUNE_CURR_BANDWIDTH	Current PI controller bandwidth (used for self-tuning of current PI controller parameters)
CURR_LP_BANDWIDTH	d/q-axis current low-pass filter band width (unit: Hz)
AC_CURR_LP_BANDWIDTH	a/b/c phase current low-pass filter band width (unit: Hz)
OLC_ANGLE_INC	Open loop angle incremental at each PWM output frequency
OLC_VOLT	Open loop voltage output (unit: voltage)
ALIGN_VOLT	Encoder align voltage (unit: voltage)
ENC_OFFSET	Encoder align offset (unit: resolution or count per revolution)
ENC_OLC_VOLT	Open loop voltage for magnetic encoder angle calibration (unit: voltage)
ENC_OLC_ANGLE_INC	Open loop angle incremental for magnetic encoder angle calibration (at each PWM output frequency)
ENC_OLC_MOTOR_REV	Forward-reverse revolutions for magnetic encoder angle calibration (unit: revolution)
FW_MAX_ID_CURR	Maximum d-axis current in field weakening control (unit: ampere)
FW_KP_GAIN	Field weakening control proportional gain (Q15 mode)
FW_KI_GAIN	Field weakening control integral gain (Q15 mode)
FW_KP_GAIN_DIV	Field weakening control proportional gain divisor (Q16 mode)
FW_KI_GAIN_DIV	Field weakening control integral gain divisor (Q16 mode)
HIGH_PWM_DUTY_CYCLE	Dynamic Adjustment of Current Sampling Position at High PWM Duty Cycle (TWO_ADC_CONVERTERS only)
MAX_DUTY_PCT	Maximum PWM Duty Cycle Setting

- Table 7 lists drive-related parameter definitions, such as dead time, shunt resistor and current amplifier gain. The corresponding parameters should be modified if other motor drive board is used.

Table 7. Driver parameter definitions

Definition	Description
VDC_RATED	DC-BUS voltage (unit: voltage)
V_SENSE_GAIN	Divider ratio of voltage feedback
ADC_REFERENCE_VOLT	ADC reference voltage (unit: voltage)
ADC_DIGITAL_SCALE_12BITS	ADC resolution
SYSTEM_CORE_CLOCK	System frequency (unit: Hz)
TMR_CLK	Timer clock frequency (unit: Hz)

Definition	Description
DEADTIME_CLK_SFT_BITS	Shift bit number of frequency divider for dead time generator
DEADTIME_NS	Dead time (unit: ns)
MIN_INTERVAL_TIME	Minimum interval time between two-phase signals during PWM shift (unit: ns)
ADC_TRIG_DELAY_TIME	Current sampling delay time (unit: us)
ADC_TRIG_LAG_TIME	ADC trigger lag time for current sensing (two-phase low-level condition) (unit: ns) (only for TWO_ADC_CONVERTERS && THREE_SHUNT)
SW_OP_INP_MODE_LEAD_TIME	OP Input Mode Switching Lead Time Prior to ADC Trigger (for chips with internal OP like M412)
MAX_CURRENT	Maximum peak phase current of motor (unit: ampere)
MIN_CURRENT	Minimum peak phase current of motor (unit: ampere)
CURRENT_SPAN_SHIFT	Number of shifts for current per-unit normalization
R_SHUNT	Shunt resistor (unit: Ω)
OP_GAIN	Current amplifier gain
CURR_OFFSET_VOLT	Zero current offset (unit: voltage)
OVER_CURRENT_SW	Software Overcurrent Threshold (unit: ampere)
DAC_VREF_SOURCE	DAC Source Selection for Internal OP Overcurrent Protection (Dedicated to chips with internal OP, e.g., M412 series)
OCP_CURRENT	Hardware Overcurrent Threshold (unit: ampere)
BUS_CURR_CMP_OCP_VOLT	Internal OP Overcurrent Protection Voltage (unit: volt) (Dedicated to chips with internal operational amplifier, e.g., M412 series)
TMR_BRK_FILTER_COUNT	Overcurrent Protection Break Input Filter Count (Dedicated to chips with break input function, e.g., M412 series)
OVER_VOLT_THRESHOLD	Over-voltage threshold point (unit: voltage)
UNDER_VOLT_THRESHOLD	Under-voltage threshold point (unit: voltage)
V0_V	Parameter V0 of the temperature and voltage linear approximation function of NTC voltage divider circuit (note [5])
T0_C	Parameter T0 of the temperature and voltage linear approximation function of NTC voltage divider circuit (note [5])
dV_dT	Parameter dV/dT of the temperature and voltage linear approximation function of NTC voltage divider circuit (note [5])
OVER_TEMP_THRESHOLD	Over-temperature threshold point (unit: Celsius degrees)
MC_ERROR_MASK	Error detection mask

Note [5]: Approximate curve equation of voltage-temperature relation is $V[V] = V0 + dV/dT[V/Celsius] * (T - T0)[Celsius]$.

- Table 8 lists definitions of motor parameters, including number of pole-pairs, encoder parameters and Hall sensor parameters.

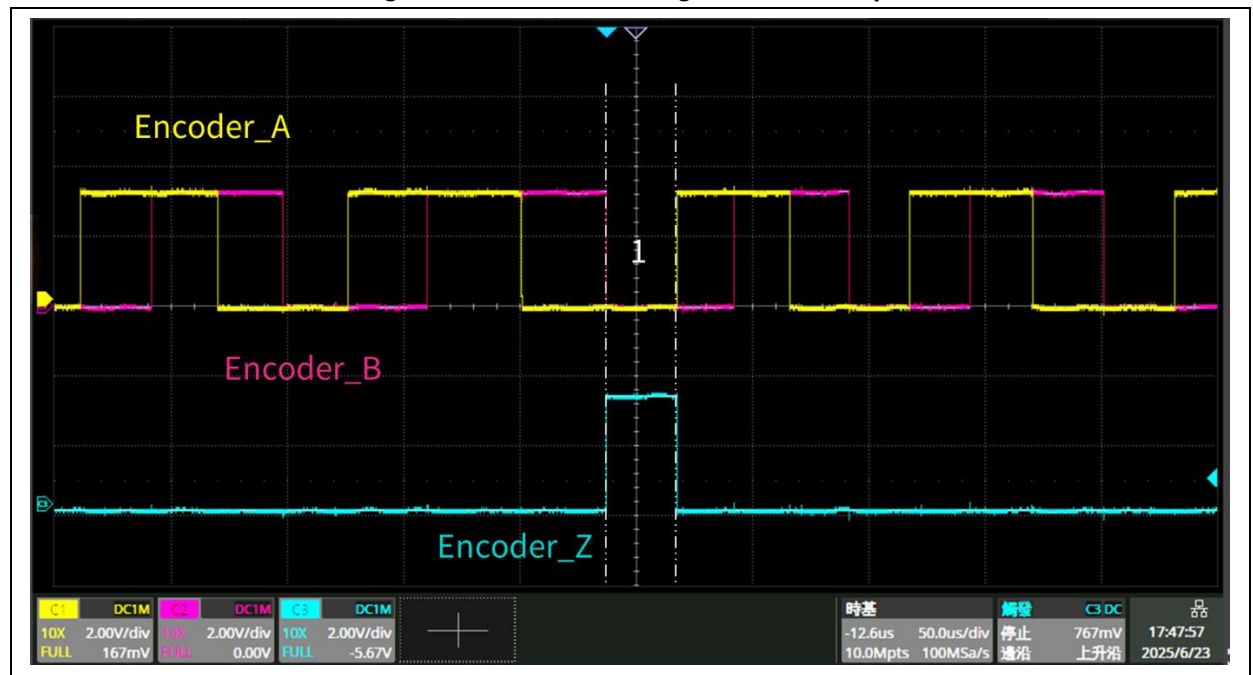
Table 8. Motor parameter definitions

Definition	Description
POLE_PAIRS	Number of pole-pairs
KE	Back EMF constant (KE) (unit: V/rpm)
LD_LQ_RATIO	Ld to Lq ratio
NOMINAL_CURRENT	Motor nominal current (unit: ampere)
ABS_ENCODER_RES	Magnetic encoder resolution (unit: bits)
ABS_ENCODER_IIF_RES	Magnetic encoder incremental interface resolution (unit: count per revolution)
ABS_ENCODER_SAMPLE_TIME	Magnetic encoder SPI interface sampling time (unit: us)
ENCODER_PPR	Pulses per revolution of magnetic encoder (unit: pulse per revolution)
ENC_IDX_COUNT	Magnetic encoder index signal width (unit: count) (note [6])
ENC_STALL_TIME	Magnetic encoder stall time (unit: ms)

Note [6]: This is applicable to ABZ mode of magnetic encoder.

Figure 8 presents the diagram of AS5047P encoder with ABZ signals. If the width gap between Z signal's rising edge and its falling edge equals to 1 count of AB signals, the ENC_IDX_COUNT is set to 1 (usually 1 or 2 counts).

Figure 8. Encoder ABZ signal relationship



2) `mc_hwio.h` file

- This file is used to configure macro definitions according to the user hardware IOs and

peripherals. It also includes the declaration of the `mc_hwio.c` file functions. Table 9 lists the configurations of peripherals such as ADC, TMR, USART and EXINT, interrupt functions and the corresponding DMA channels.

Table 9. Configuration of peripherals (AT32F413RCT7)

Peripheral	Interrupt function	DMA channel	Description
ADC1	N/A	DMA1_CH1	ADC1 ordinary channel conversion (BUS voltage, MOS temperature, external speed/torque command, three-phase BEMF voltage and BUS current)
ADC1	ADC1_2_IRQn	N/A	ADC1 preempted channel conversion (phase current or bus current)
TMR1	TMR1_OVF_TMR10_IRQn	N/A	FOC control interrupt
TMR1	TMR1_BRK_TMR9_IRQn	N/A	PWM output disable
TMR1_CH4	N/A	DMA1_CH4	Set sampling time point configuration in one-shunt mode
TMR5	N/A	N/A	Magnetic encoder incremental interface counting mode
TMR3	TMR3_GLOBAL_IRQHandler	N/A	Magnetic encoder incremental interface capture mode
TMR4	TMR4_GLOBAL_IRQHandler	N/A	Magnetic encoder SPI interface sampling
TMR11	TMR1_TRG_HALL_TMR11_IRQHandler	N/A	Speed control loop interrupt
USART1_TX	N/A	DMA1_CH2	USART1 TX data
USART1_RX	N/A	DMA1_CH3	USART1 RX data
EXINT	EXINT4_IRQn	N/A	Encoder zero-position calibration
EXINT	EXINT15_10_IRQn	N/A	START/STOP button (start/stop)

3) *mc_hwio.c* file

- This file is mainly configured the MCU peripherals based on user hardware, such as TMR, ADC, DMA and GPIO. It also includes some basic control functions such as button, LED, DIP switch. Refer to Table 10 for details.

Table 10. Peripheral configuration related functions

Function	Description
nvic_config	Interrupt priority configuration
tmr_pwm_init	PWM timer, crm clock, GPIO and DMA configuration
encoder_time_init	Magnetic encoder incremental interface timer, crm clock, GPIO and EXINT configuration
encoder_capture_timer_init	Magnetic encoder incremental interface capture timer, crm clock and GPIO (for FOC only)
magnetic_encoder_timer_init	Magnetic encoder SPI interface sampling timer configuration (synchronized with PWM clock)
adc_ordinary_config	ADC ordinary channel ADC, DMA and GPIO configuration
adc_preempt_config	ADC preempted channel ADC, DMA and GPIO configuration
speed_timer_init	Speed control timer and crm clock configuration
uart_init	UART crm clock, GPIO and UART configuration
button_exint_init	Button interrupt event EXINT configuration
led_config	LED GPIO initialization and crm clock configuration
led_on	LED ON
led_off	LED OFF
led_toggle	LED toggle (from ON to OFF, or from OFF to ON)
led_init	LED initialization configuration
led_blink	LED blinks
mode_switch_init	DIP switch configuration
motor_parameter_ID_config	Configure TMR and ADC relevant to motor winding parameter identification
get_int_vref_cal_ratio	Detect VDDA reference voltage ratio, used for ADC value calibration
opa_init	Busbar Current Amplification OP Configuration (AT32M412 and other chips with internal OP only)
opa_calibration	Busbar Current Amplification OP Calibration (AT32M412 and other chips with internal OP only)
ocp_cmp_config	Busbar Current Protection CMP Configuration (AT32M412 and other chips with internal CMP only)

4) **mc_isr.c** file

- This file includes interrupt handler functions for ADC, TMR and SYSTICK interrupts, etc. Refer to Table 11.

Table 11. Motor control related interrupt functions (AT32F413RCT7)

Function	Description
ADVTMR_PWM_CYCLE_IRQ (TMR1_OVF_TMR10_IRQHandler)	TIMER1 update interrupt function: get angle, vector control, open loop control, voltage loop control, current loop control

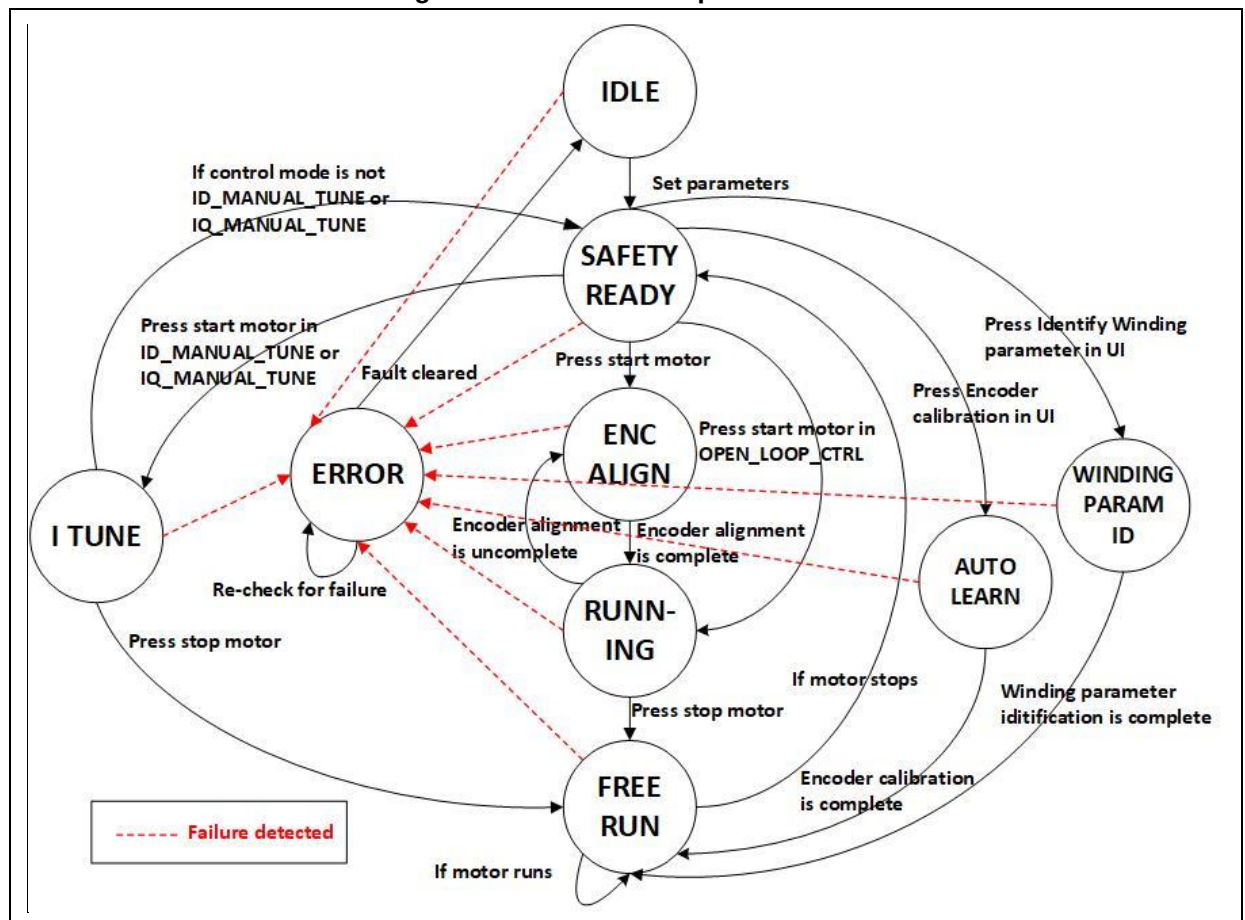
ADVTMR_PWM_BRK_IRQ (TMR1_BRK_TMR9_IRQHandler)	TMR1 brake input interrupt function: PWM output disable
ADC_SHUNT_SAMP_READY_IRQ (ADC1_2_IRQHandler)	ADC1 interrupt function: current sampling complete
ENCODER_CAPTURE_IRQ (TMR3_GLOBAL_IRQHandler)	Magnetic encoder incremental interface capture interrupt function
EXINT_ENCODER_IDX_IRQ (EXINT4_IRQHandler)	External IO interrupt function: magnetic encoder incremental interface zero-position calibration
SYNC_TIMER_CH_IRQ (TMR4_GLOBAL_IRQHandler)	Magnetic encoder SPI interface sampling interrupt function
SPEED_LOOP_TIMER_IRQ (TMR1_TRG_HALL_TMR11_IRQHandler)	Speed control loop interrupt function, get speed
SysTick_Handler	System interrupt function (1 ms): includes state machine program
BUTTON_EXINT_IRQ (EXINT15_10_IRQHandler)	External IO interrupt function: START/STOP button (start/stop motor)

4 State machine

4.1 Description of state

Figure 9 shows the state machine of the application example of motor control library. The initial settings of each state are constantly checked by the Main program. When the state changes, the initial settings are set for the new state. In addition, the state machine runs in the SysTick Handler interrupt function every millisecond to ensure that it is checked in real time. Each state is checked for fault; once detected, the state machine enters “Error” state and stop driving to avoid damage to the motor or motor drive board, and the motor stops until the fault is cleared.

Figure 9. State machine process flow



It includes Idle, Safety ready, Running, Free run, I_tune, Enc_align and Error, as detailed below.

Idle

It is the initial state of state machine, and motor idles in this state. The state machine returns to “Idle” state when the error condition is released.

Safety ready

This state indicates that the motor can be started up safely after all parameters are set and checked and the current offset is obtained in the “Idle” state.

Running

The motor is running in this state. Through the UI software, users can on-line adjust parameters (target speed, target current, etc.) or stop the motor.

Free run

In this state, the drive stops voltage output, and the motor speed will slowly down to zero. Program remains in this state until the motor stops completely, and then returns to the “Safety ready” state.

I_tune

This state is used for adjusting PID controller parameters. In this state, users can adjust the target current, current loop KP and KI values through UI software. A step current is generated in this state, based on which parameters are adjusted.

Enc_align

This represents the zero-position calibration state of an incremental encoder. With the UI interface, the user can enable zero-position calibration feature. Typically, after a driver reset, it is necessary to activate zero-position calibration. If no such action implemented, an auto zero-position calibration will be triggered before motor startup.

Winding param Id

This refers to the state of motor winding parameter identification. Users can enable this feature via UI interface. The winding parameter identification feature is often used for tuning a new motor. The tuned parameters can be used for sensorless vector control and current PI controller parameters self-tuning.

Auto learn

This refers to a self-learning feature in our motor control library. In this demo, it is used in magnetic encoder deviation angle self-calibration.

Error

The state machine jumps to “Error” state in case of any failures.

Table 12 lists each state in details and state switching conditions.

Table 12. Initial state and state switching conditions

Initial state	End state	State switching conditions
IDLE	SAFETY REDAY	The control source is the UI tool, and initial parameters are set.
		The control source is the motor control board, POTENTIOMETER command is given less than the default, and initial parameters are set.
SAFETY REDAY	ENC ALIGN	When the control mode is not ID_MANUAL_TUNE nor IQ_MANUAL_TUNE, click START MOTOR (UI tool) or START/STOP button on the motor control board.
	I TUNE	When the control mode is ID_MANUAL_TUNE or IQ_MANUAL_TUNE, click START MOTOR (UI tool) or START/STOP button on the motor control board.
	RUNNING	When the control mode is OPEN_LOOP_CTRL, click START MOTOR (UI tool) or START/STOP button on the motor control board.
	WINDING_PARAM_ID	Click “Identify Winding parameter” (UI tool).
	AUTO_LEARN	Click “Encoder calibration” (UI tool).
ENC ALIGN	RUNNING	Control mode is not OPEN_LOOP_CTRL and encoder alignment is completed.

RUNNING	ENC ALIGN	Control mode is not OPEN_LOOP_CTRL and encoder alignment is not completed.
	FREE_RUN	Click STOP MOTOR (UI tool) or START/STOP button on the motor control board.
FREE_RUN	SAFETY REDAY	Motor stops.
	FREE_RUN	Motor runs.
I TUNE	SAFETY REDAY	Control mode is not ID_MANUAL_TUNE nor IQ_MANUAL_TUNE.
	FREE_RUN	Click STOP MOTOR (UI tool) or START/STOP button on the motor control board.
WINDING_PARAM_ID	FREE_RUN	Motor winding parameter identification is completed.
AUTO_LEARN	FREE_RUN	Magnetic encoder deviation angle calibration is completed.
ERROR	IDLE	Fault is cleared.

5 How to use demo project

For details, refer to *AN0196_AT32_PMSM_FOC_Magnetic_Encoder_Quick_Start_Guide*.

6 Revision history

Table 13. Document revision history

Date	Version	Revision note
2025.10.09	2.0.0	Initial release.

IMPORTANT NOTICE – PLEASE READ CAREFULLY

Purchasers are solely responsible for the selection and use of ARTERY's products and services; ARTERY assumes no liability for purchasers' selection or use of the products and the relevant services.

No license, express or implied, to any intellectual property right is granted by ARTERY herein regardless of the existence of any previous representation in any forms. If any part of this document involves third party's products or services, it does NOT imply that ARTERY authorizes the use of the third party's products or services, or permits any of the intellectual property, or guarantees any uses of the third party's products or services or intellectual property in any way.

Except as provided in ARTERY's terms and conditions of sale for such products, ARTERY disclaims any express or implied warranty, relating to use and/or sale of the products, including but not restricted to liability or warranties relating to merchantability, fitness for a particular purpose (based on the corresponding legal situation in any unjudicial districts), or infringement of any patent, copyright, or other intellectual property right.

ARTERY's products are not designed for the following purposes, and thus not intended for the following uses: (A) Applications that have specific requirements on safety, for example: life-support applications, active implant devices, or systems that have specific requirements on product function safety; (B) Aviation applications; (C) Aerospace applications or environment; (D) Weapons, and/or (E) Other applications that may cause injuries, deaths or property damages. Since ARTERY products are not intended for the above-mentioned purposes, if purchasers apply ARTERY products to these purposes, purchasers are solely responsible for any consequences or risks caused, even if any written notice is sent to ARTERY by purchasers; in addition, purchasers are solely responsible for the compliance with all statutory and regulatory requirements regarding these uses.

Any inconsistency of the sold ARTERY products with the statement and/or technical features specification described in this document will immediately cause the invalidity of any warranty granted by ARTERY products or services stated in this document by ARTERY, and ARTERY disclaims any responsibility in any form.

© 2025 ARTERY Technology – All Rights Reserved